WEEK 3 · MACHINE LEARNING · BS-V

Decision Trees

Learning to ask the right question first

The ID3 Algorithm — Entropy, Information Gain, and a tree built by hand



Dr Muhammad Ismail Mangrio
Department of Computer Science · Sukkur IBA University

THE PROBLEM

It is 6 a.m. Do you play today?

You are the groundskeeper at a tennis club.

- Set up the courts and nobody comes → wasted morning.
- Cancel and the sun comes out → angry members.
- You have 14 days of records: the weather, and whether play happened.
- You want a rule you can hand to the new intern.

What kind of rule?

Not a probability. Not a black box.
A short list of questions anyone can follow:

What is the outlook?

If sunny — how humid?

If rainy — how windy?

→ Play / Don't play

ROADMAP

Where we are going

1 The tree

Anatomy: root, decision nodes, branches, leaves

2 The question

Which attribute should we split on first?

3 Entropy

Measuring how mixed a group of examples is

4 Information Gain

How much a split cleans up that mess

5 ID3 by hand

Build the full tennis tree, split by split

6 Limits

Where ID3 breaks — and what comes next

CONCEPT

Anatomy of a decision tree

Root node

The first question. Chosen because it separates the classes better than any other attribute.

Decision node

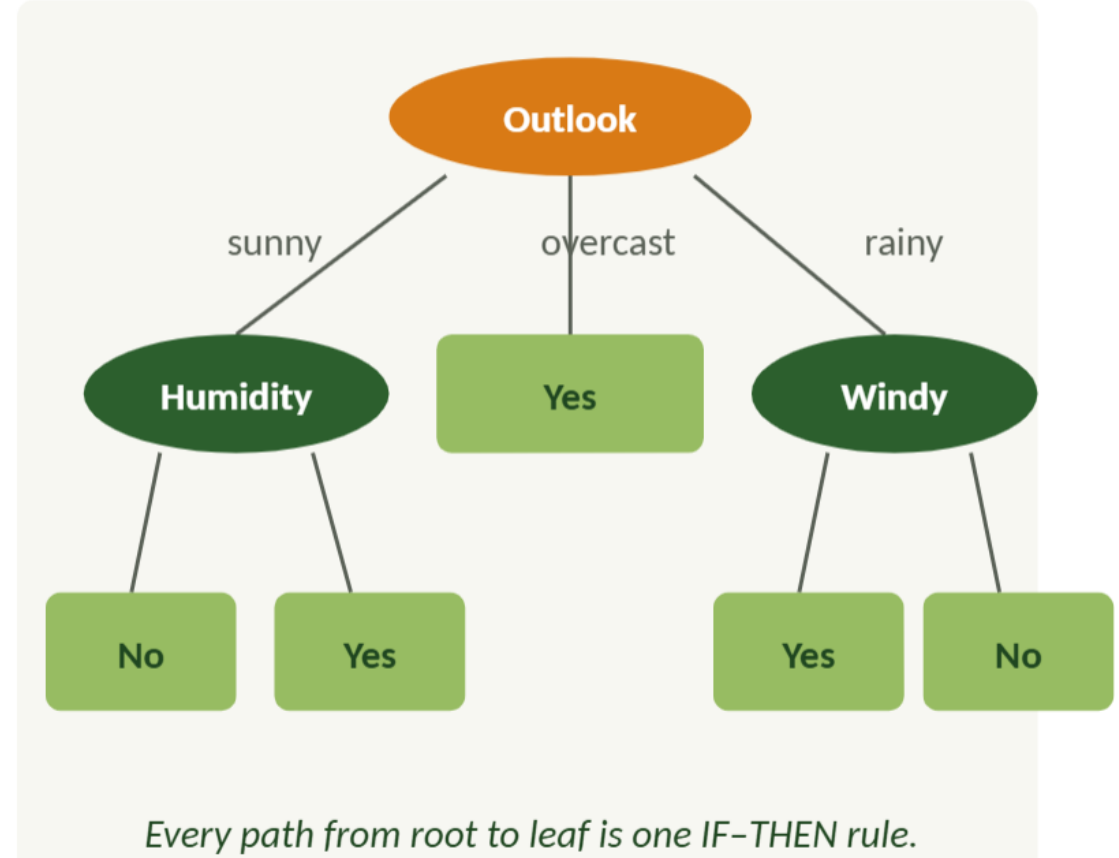
An internal test on one feature — 'Humidity = ?'

Branch

One possible answer to that test. A rule.

Leaf node

No more questions. This is the predicted outcome.



Every path from root to leaf is one IF–THEN rule.

MOTIVATION

Why anyone still uses trees in 2026

1 Explainable

A bank must tell you why your loan was refused. A tree prints the reason as a sentence.

2 No scaling needed

No normalisation, no dummy variables required. Mixed data types are fine.

3 Handles non-linearity

Splits carve the space into boxes — no straight-line assumption.

4 The building block

Random Forest and XGBoost are just many trees. Get this right and Week 13 is easy.

LANDSCAPE

Two families of tree algorithm

ID3 / C4.5

Iterative Dichotomiser 3

- Splits on Entropy + Information Gain
- Multi-way splits (one branch per value)
- Designed for categorical attributes
- Classification only
- Our focus today — done by hand

CART

Classification and Regression Trees

- Splits on the Gini Index
- Always binary splits
- Handles continuous attributes natively
- Classification and regression
- What scikit-learn actually runs

Same idea, different ruler for measuring 'messiness'.

DATA

The evidence: 14 days of records

#	Outlook	Temp	Humidity	Windy	Play
1	Sunny	Hot	High	Weak	No
2	Sunny	Hot	High	Strong	No
3	Overcast	Hot	High	Weak	Yes
4	Rainy	Mild	High	Weak	Yes
5	Rainy	Cool	Normal	Weak	Yes
6	Rainy	Cool	Normal	Strong	No
7	Overcast	Cool	Normal	Strong	Yes
8	Sunny	Mild	High	Weak	No
9	Sunny	Cool	Normal	Weak	Yes
10	Rainy	Mild	Normal	Weak	Yes
11	Sunny	Mild	Normal	Strong	Yes
12	Overcast	Mild	High	Strong	Yes
13	Overcast	Hot	Normal	Weak	Yes
14	Rainy	Mild	High	Strong	No

The class balance

9 Yes **5** No

Total = 14 examples, 4 attributes

Ask yourself first

Scan the Outlook column. Every single Overcast day is a Yes.

That column already contains a perfect rule. Entropy is how we let a computer notice that.

First, we need a way to measure mess.

Before we can choose the best question, we must be able to score how mixed-up a group of examples is.

INTUITION

Entropy = how surprised will you be?

Bag A

5 red · 5 blue

Pick one blindly and you have no idea what you will get. Maximum surprise.

Entropy = 1.0 (maximum)

Bag B

10 red · 0 blue

Pick one blindly and you know exactly what you will get. No surprise at all.

Entropy = 0 (pure)

The formula

$$H(S) = -\Sigma\ p_i \cdot \log_2(p_i)$$

$H(S)$	entropy of the set S
p_i	share of class i inside S
$\log_2$	measured in bits
$-\Sigma$	summed over all classes

WORKED EXAMPLE · STEP 1

Entropy of the whole dataset

Step 1 — Proportions

$$\text{Total} = 14$$

$$p(\text{Yes}) = 9/14 = 0.643$$

$$p(\text{No}) = 5/14 = 0.357$$

Step 2 — Substitute

$$H = -0.643 \times \log_2 0.643 - 0.357 \times \log_2 0.357$$

$$H = -0.643 \times (-0.637) - 0.357 \times (-1.485)$$

Step 3 — Add up

$$H = 0.410 + 0.530$$

$$H(S) = 0.940 \text{ bits}$$



0.940 out of a possible 1.0 — this set is almost as mixed as it can be. There is a lot of room for a good question to help.

CONCEPT

Information Gain: what a question buys you

$$\text{Gain}(A) = \text{Entropy}(S) - I(A) \qquad I(A) = \sum (p_i+n_i)/(p+n) \times \text{Entropy}(A_i)$$

mess before the split – weighted average mess after the split

1

Why weighted?

A branch holding 7 examples matters more than a branch holding 2. We weight each child's entropy by its share of the parent.

2

What is a good gain?

High gain = the split produced purer groups. ID3 is greedy: at every node it simply takes the largest gain available right now.

!

The catch

Gain is biased toward attributes with many distinct values. Split on Student_ID and gain is maximal — and the tree is useless. C4.5 fixes this with Gain Ratio.

THE STEP STUDENTS GET WRONG

Why the average after a split must be weighted

A everyday example: two lab sections

```
Section A: 10 students, average 40 marks
Section B:  2 students, average 90 marks
```

Plain average: **(40 + 90) / 2 = 65** ← *wrong*

Weighted: **(10/12)(40) + (2/12)(90) = 48.3**

Two students must not shout as loudly as ten.

The same trap inside a tree

Imagine a split that gives one branch with 13 messy rows (H = 0.99) and one branch with a single row (H = 0).

Plain average = 0.495 — looks like a great split.
Weighted = (13/14)(0.99) = 0.919 — barely any improvement. The weighted number is the honest one.

Reading the formula out loud

$$I(A) = \Sigma \ (p_i+n_i)/(p+n) \times Entropy(A_i)$$

$p_i + n_i$	how many examples landed in branch i
$p + n$	how many examples entered the node
the fraction	branch i's share of the node — its voting power
$Entropy(A_i)$	how messy branch i still is
Σ	add up every branch's contribution

How much mess is left — big branches count more.

WORKED, ONE BRANCH AT A TIME

The weighted average on our own split

Split the 14 days on Outlook. Each branch contributes its entropy in proportion to its size.

Sunny

5 of 14 days

share of node	5/14 = 0.357
branch entropy	0.971
contribution	0.357 × 0.971

= 0.347

Rainy

5 of 14 days

share of node	5/14 = 0.357
branch entropy	0.971
contribution	0.357 × 0.971

= 0.347

Overcast

4 of 14 days

share of node	4/14 = 0.286
branch entropy	0.000
contribution	0.286 × 0.000

= 0.000

I(Outlook) = 0.347 + 0.347 + 0.000 = **0.693** → Gain = 0.940 − 0.693 = **0.247**

ALGORITHM

The ID3 recipe

1. Compute Entropy(S) for the current set of examples.
2. For every remaining attribute A: entropy of each of its values, then the weighted average I(A), then Gain(A) = Entropy(S) − I(A).
3. Split on the attribute with the highest gain. It becomes this node.
4. Repeat on each child subset, using the attributes not yet used on that path.

Stop when: the subset is pure, no attributes remain, or the subset is empty.

Remember

ID3 is greedy.

It never looks two questions ahead and it never undoes a split it has made.

So it finds a good tree — not provably the smallest one.

Finding the optimal tree is NP-hard.

Round 1: which of the four goes at the root?

Four candidates. Four gains. One winner.

ROUND 1

Candidate 1 — Outlook, worked in full

Outlook	p (Yes)	n (No)	Entropy
Sunny	2	3	0.971
Rainy	3	2	0.971
Overcast	**4**	**0**	**0.000**

Overcast is already pure — entropy 0. That branch is finished before we even start.

Weighted average

```
I = (5/14)(0.971) + (5/14)(0.971) + (4/14)(0)
  = 0.347 + 0.347 + 0
  = 0.693
```

The gain

```
Entropy(S)          0.940
−  I(Outlook)       0.693
```

Gain(Outlook) = **0.247**

Reading it in plain words: asking about the outlook removes about a quarter of a bit of our uncertainty about whether play happens. Now check whether any other question removes more.

ROUND 1

Candidates 2, 3 and 4 — same three lines

Temperature

Value	p	n	H
Hot	2	2	1.000
Mild	4	2	0.918
Cool	3	1	0.811

I = 0.911

Gain = 0.029

Humidity

Value	p	n	H
High	3	4	0.985
Normal	6	1	0.591

I = 0.788

Gain = 0.152

Windy

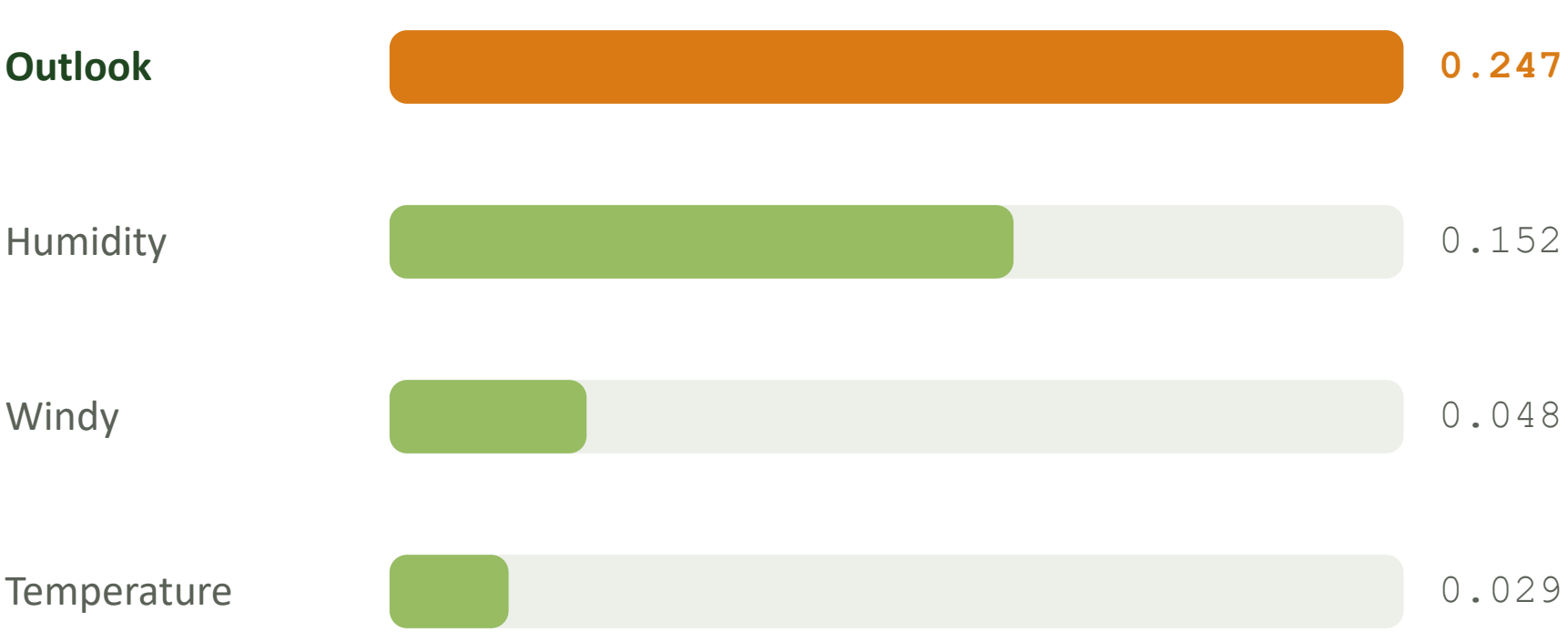
Value	p	n	H
Strong	3	3	1.000
Weak	6	2	0.811

I = 0.892

Gain = 0.048

ROUND 1 · DECISION

The verdict for the root



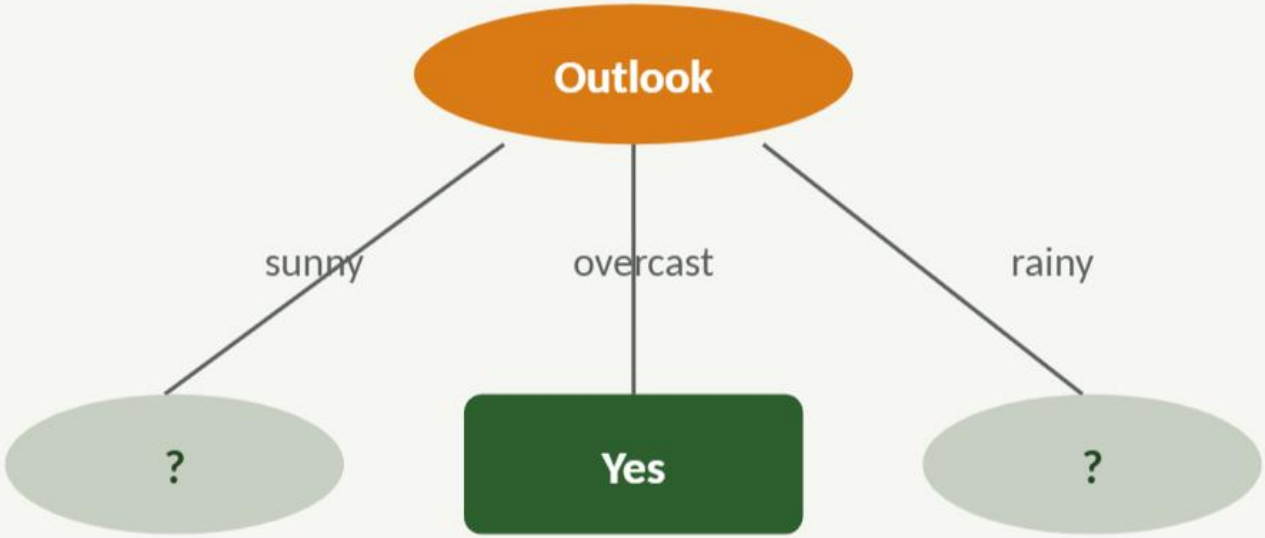
ROOT

Outlook

Note how far ahead Outlook is. Temperature — the attribute students most often guess — is almost worthless here.

PROGRESS

The tree after one split



Now recurse

The overcast branch is closed. Two subsets remain, and each is a smaller version of the original problem.

So we run exactly the same three lines again — but only on the 5 sunny rows, and only with the 3 attributes we have not used on that path.

ROUND 2 · SUNNY

The sunny branch: 5 rows, 2 Yes / 3 No

$$\text{Entropy}(S_{\text{sunny}}) = -(2/5)\log_2(2/5) - (3/5)\log_2(3/5) = 0.971$$

Humidity

Value	p	n	H
High	0	3	0
Normal	2	0	0

$I = 0.000$

Gain = 0.971

Temperature

Value	p	n	H
Hot	0	2	0
Mild	1	1	1.000
Cool	1	0	0

$I = 0.400$

Gain = 0.571

Windy

Value	p	n	H
Strong	1	1	1.000
Weak	1	2	0.918

$I = 0.951$

Gain = 0.020

Humidity splits the sunny days perfectly — both children are pure, so that branch is finished.

ROUND 2 · RAINY

The rainy branch: 5 rows, 3 Yes / 2 No

$$\text{Entropy}(S_{\text{rainy}}) = -(3/5)\log_2(3/5) - (2/5)\log_2(2/5) = 0.971$$

Windy

Value	p	n	H
Strong	0	2	0
Weak	3	0	0

$I = 0.000$

Gain = 0.971

Humidity

Value	p	n	H
High	1	1	1.000
Normal	2	1	0.918

$I = 0.951$

Gain = 0.020

Temperature

Value	p	n	H
Cool	1	1	1.000
Mild	2	1	0.918

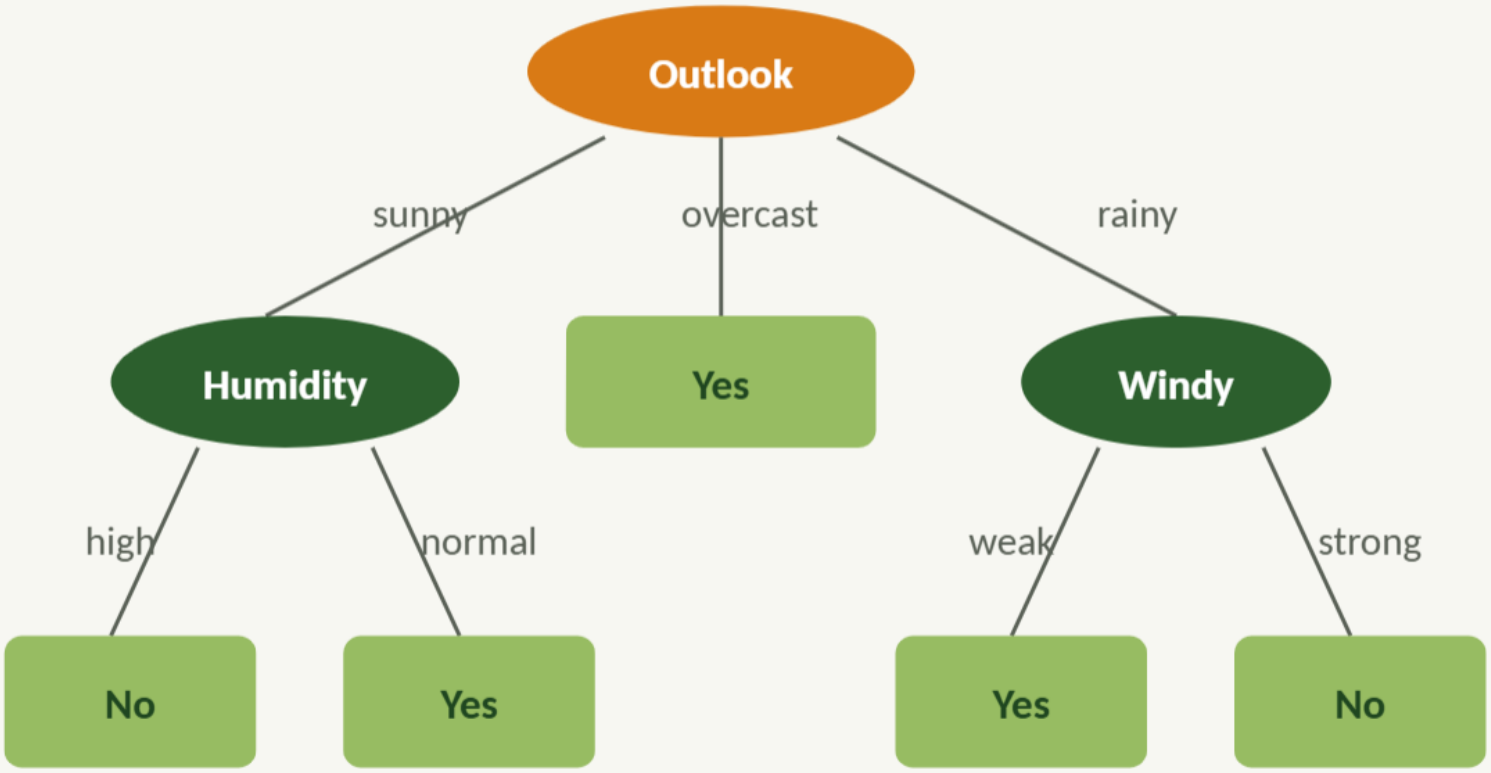
$I = 0.951$

Gain = 0.020

Notice: Humidity was the hero on the sunny branch and is useless here. The best question depends entirely on where you already are in the tree.

RESULT

The finished tree



Depth 2 · 5 leaves · Temperature never used at all.

Read it as rules

Sunny AND high humidity → No

Sunny AND normal humidity → Yes

Overcast → Yes

Rainy AND weak wind → Yes

Rainy AND strong wind → No

Five sentences. Hand them to the intern.

CHECK YOURSELF

Your turn: a new morning arrives

Tomorrow's forecast

```
Outlook  = Rainy     Humidity = High
Temp     = Cool      Windy    =
Strong
```

Answer these three

- Which path does this day take?
- Which two attributes did the tree ignore, and why?
- What would the tree predict if Windy were Weak?

Trace it

1. Root: Outlook = Rainy → go right
2. Node: Windy = Strong → go right
3. Leaf reached: No
4. **Prediction: don't set up courts**

The tree only asks what it needs.

Now the uncomfortable part.

The tree we just built is perfect on the data it has already seen. That is not a compliment.

LIMITATIONS

Four things ID3 cannot do

Overfitting

With enough attributes ID3 grows until every leaf is pure — memorising noise. 100% training accuracy, poor test accuracy.

Fix: Pre-pruning (max depth, min samples) or post-pruning.

Continuous attributes

ID3 needs discrete values. 'Temperature = 27.4 °C' gives one branch per reading.

Fix: Threshold splits: Temp ≤ 25? — this is what CART does.

Multi-valued bias

Information Gain rewards attributes with many values, even meaningless ones.

Fix: Gain Ratio (C4.5) normalises by the attribute's own entropy.

Missing values & instability

ID3 has no rule for a missing cell, and changing a few rows can redraw the whole tree.

Fix: Surrogate splits; ensembles (Random Forest) average the instability away.

WHERE INFORMATION GAIN BREAKS

A worked failure: what if Roll_No were an attribute?

Add one harmless column

Roll_No	Outlook	Play
D1	Sunny	No
D2	Sunny	No
D3	Overcast	Yes
D4	Rainy	Yes
…	…	…
D14	Rainy	No

Every value appears exactly once, so every branch holds one row — and one row is always pure.

The arithmetic

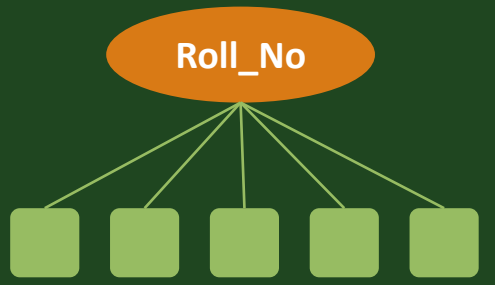
```
Entropy(D1)   = 0
Entropy(D2)   = 0
      …
Entropy(D14)  = 0

I(Roll_No)
 = 14 × (1/14) × 0
 = 0.000
```

Gain = 0.940

The largest gain possible — far above Outlook's 0.247. ID3 picks Roll_No as the root.

The tree it builds



14 branches, 14 leaves

100% accurate on the training data. On day 15 the roll number is one it has never seen, so no branch matches and the tree predicts nothing. It memorised; it learned nothing.

How the failure is fixed

1

Split Information

Measure how widely the attribute itself scatters the data:
$SplitInfo(A) = -\Sigma\ (|S_i|/|S|)\ \log_2(|S_i|/|S|)$
For Roll_No: $SplitInfo = \log_2(14) = 3.807$ — very high.

2

Gain Ratio (C4.5)

Divide the gain by that scatter: GainRatio = Gain / SplitInfo.
Roll_No: 0.940 / 3.807 = 0.247 — no longer spectacular. C4.5 also requires a candidate to have above-average gain first, which filters identifier columns out entirely.

3

The engineer's fix

Drop identifier columns before training. No algorithm should be asked to decide whether a roll number is meaningful — you already know it is not.

Rule of thumb: if a feature is unique per row, it cannot generalise — no matter how good its gain looks.

LOOKING AHEAD

Entropy or Gini — does it matter?

	Entropy (ID3, C4.5)	Gini Index (CART)
Formula	$- \Sigma\ p_i \log_2 p_i$	$1 - \Sigma\ p_i^2$
Range (2 classes)	0 to 1	0 to 0.5
Cost	Logarithms — slower	Squares only — faster
Split choice	Agree ~98% of the time	Agree ~98% of the time
Used by	Classic ID3 / C4.5	scikit-learn default

Practical answer: pick either. The impurity measure is rarely what decides whether your tree is any good — depth, pruning and data quality are.

SIX THINGS TO LEAVE WITH

Key takeaways

1. A decision tree is a sequence of questions; every root-to-leaf path is an IF–THEN rule.

2. Entropy measures impurity: 0 = pure, 1 = a 50/50 coin flip (two classes).

3. Information Gain = entropy before, minus the size-weighted entropy after. Big branches count more.

4. ID3 is greedy and recursive: take the highest gain now, then repeat on each child.

5. The best attribute is local — Humidity won under sunny, Windy won under rainy.

6. Unrestrained trees overfit — and a unique-per-row column like Roll_No maximises gain while learning nothing.

LAB 3 & WEEK 4

What happens next

In Lab 3 this week

- Load the cleaned CSV your Lab 2 produced
- Fit DecisionTreeClassifier with criterion='entropy'
- Plot the tree and read the splits out loud
- Vary max_depth and watch train vs test accuracy diverge
- Compare Gini against entropy on the same split

Before the next lecture

- Build the ID3 tree by hand for the same data with the Overcast rows removed — does the root change?
- Compute the Gini index of the root node and compare it to 0.940
- Read Mitchell Ch. 3, §3.4 to §3.7

Questions: office hours, or the course forum on the LMS.